

# Information and Network Security

## Study Guide

**Dr. Rahul Sharma**  
CSE, PIT  
Parul University

|                                  |   |
|----------------------------------|---|
| 5.1 The Euclidean Algorithm..... | 3 |
|----------------------------------|---|

## 5.1 The Euclidean Algorithm

The Euclidean Algorithm is an efficient method for computing the **greatest common divisor (GCD)** of two integers. The GCD of two numbers is the largest number that divides both of them without leaving a remainder. This algorithm is one of the oldest known, attributed to the ancient Greek mathematician Euclid around 300 BCE, and it remains a foundational concept in number theory and computer science.

The algorithm is based on the principle that the GCD of two numbers also divides their difference. More precisely, for any two positive integers **a** and **b**, where **a > b**, the GCD of **a** and **b** is the same as the GCD of **b** and **a mod b** (where “a mod b” is the remainder when **a** is divided by **b**). This process is repeated until the remainder becomes zero. At that point, the non-zero divisor is the GCD.

### Steps of the Euclidean Algorithm:

- Given two integers **a** and **b**, where  $a \geq b$ .
- Divide **a** by **b**, and let the remainder be **r**.
- Replace **a** with **b** and **b** with **r**.
- Repeat the process until  $r = 0$ .
- When  $r = 0$ , the GCD is the current value of **b**.

### Example:

Find the GCD of 252 and 105:

- $252 \div 105 = 2$  remainder 42  $\rightarrow$  replace (252, 105) with (105, 42)
- $105 \div 42 = 2$  remainder 21  $\rightarrow$  replace (105, 42) with (42, 21)
- $42 \div 21 = 2$  remainder 0  $\rightarrow$  stop

The GCD of 252 and 105 is 21.

The Euclidean Algorithm is not only simple but also very fast, even for very large numbers. It forms the basis for many applications, including simplifying fractions, cryptographic algorithms like RSA, and solving Diophantine equations.

The Euclidean Algorithm is a powerful and time-tested method for determining the greatest common divisor of two integers, playing a crucial role in both theoretical and applied

mathematics.

## Next Steps

- Implement the Euclidean Algorithm in any programming language to compute GCD.
- Extend to multiple numbers by computing GCD of arrays or sequences.
- Learn the Extended Euclidean Algorithm to find integers  $x$  and  $y$  such that  $ax + by = \gcd(a, b)$ .
- Apply in real-world contexts like simplifying fractions and solving modular equations.
- Understand its role in cryptography, especially RSA and modular inverses.
- Analyse time complexity – it runs in  $O(\log \min(a, b))$ , making it very efficient.
- Practice problems on coding platforms like LeetCode, HackerRank, and Codeforces.
- Explore theoretical applications in number theory and linear Diophantine equations.

### References:

#### Books:

- Rosen, K. H. (2012). Discrete Mathematics and Its Applications (7th ed.). McGraw-Hill.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- Burton, D. M. (2011). Elementary Number Theory (7th ed.). McGraw-Hill.

#### Online Resources:

- Khan Academy: <https://www.khanacademy.org>
- GeeksforGeeks: <https://www.geeksforgeeks.org/euclidean-algorithms-gcd/>
- Brilliant: <https://brilliant.org/wiki/euclidean-algorithm/>

## 5.2 Modular Arithmetic, Finite Fields of the Form

The Euclidean Algorithm is an efficient method for computing the greatest common divisor (GCD) of two integers. The GCD of two numbers is the largest number that divides both of them without leaving a remainder. This algorithm is one of the oldest known, attributed to the ancient Greek mathematician Euclid around 300 BCE, and it remains a foundational concept in number theory and computer science.

Modular arithmetic is a system of arithmetic for integers, where numbers wrap around upon reaching a certain value called the modulus. It is sometimes referred to as “clock arithmetic” because of its resemblance to how clocks reset every 12 or 24 hours. In modular arithmetic, two numbers are said to be congruent modulo  $n$  if they leave the same remainder when divided by  $n$ . This is denoted as:  $a \equiv b \pmod{n}$ . For example,  $17 \equiv 5 \pmod{12}$  because both 17 and 5 leave the same remainder when divided by 12.

Modular arithmetic is fundamental in various branches of mathematics and is especially important in cryptography, where secure communication relies on operations in modular systems. Many cryptographic algorithms, including RSA and AES, utilize modular operations to ensure data security.

### Finite Fields (Galois Fields)

A finite field, also known as a Galois Field (GF), is a set containing a finite number of elements on which you can perform addition, subtraction, multiplication, and division (excluding division by zero) such that the results always stay within the field. For a set to be a field, it must satisfy all the field axioms like associativity, distributivity, and the existence of additive and multiplicative identities and inverses.

The simplest type of finite field is denoted as  $GF(p)$ , where  $p$  is a prime number. In  $GF(p)$ , the elements are the integers  $\{0, 1, 2, \dots, p-1\}$ , and all arithmetic operations are performed modulo  $p$ . Since  $p$  is prime, every non-zero element in  $GF(p)$  has a multiplicative inverse, which is necessary for division to be well-defined in the field.

For example, in  $GF(5)$ , the elements are  $\{0, 1, 2, 3, 4\}$ , and:

- $2+4=6\equiv 1 \pmod{5}$   $2 + 4 = 6 \equiv 1 \pmod{5}$
- $3\times 4=12\equiv 2 \pmod{5}$   $3 \times 4 = 12 \equiv 2 \pmod{5}$
- The multiplicative inverse of 3 is 2 because  $3\times 2=6\equiv 1 \pmod{5}$   $3 \times 2 = 6 \equiv 1 \pmod{5}$

Finite fields are critical in cryptography, error-correcting codes, **and** digital signal processing, where computations must be limited to a finite set of values. In more advanced cryptographic applications like AES, fields of the form  $GF(2^n)$  are used, where elements are polynomials over  $GF(2)$  and operations are performed modulo an irreducible polynomial.

Thus, modular arithmetic and finite fields form the mathematical backbone of many modern technologies and encryption standards.

## Next Steps

- **Study Extended Euclidean Algorithm:** Learn how to compute modular inverses using the Extended Euclidean Algorithm, which is crucial for division in  $GF(p)$  and many cryptographic algorithms.
- **Explore Fields of the Form  $GF(2^n)$ :** Move beyond prime fields to study finite fields with  $2^n$  elements, used in AES. Understand polynomial representation and arithmetic modulo irreducible polynomials.
- **Practice Polynomial Arithmetic in  $GF(2)$ :** Perform addition, multiplication, and modulo reduction of binary polynomials—this is the foundation for encryption algorithms like AES and error-correcting codes.
- **Implement Modular Arithmetic in Code:** Write simple programs (in Python, C, or Java) to perform modular operations and field arithmetic to solidify theoretical knowledge through practice.
- **Connect with Cryptographic Applications:** Study how modular arithmetic and finite fields are used in real-world cryptography, such as RSA (mod  $n$  with large primes) and AES (in  $GF(2^8)$ ).

**References:**

- **"Cryptography and Network Security: Principles and Practice" by William Stallings**  
Covers cryptographic algorithms, including modular arithmetic and finite fields, with real-world applications like RSA and AES.
- **"An Introduction to Mathematical Cryptography" by Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman** A solid introduction to modular arithmetic, finite fields, and cryptographic protocols.
- **"Introduction to the Theory of Error-Correcting Codes" by Vera T. Phan and Shigeo Morimoto** A detailed study of finite fields, polynomial arithmetic, and error-correcting codes, which relate closely to cryptography.
- **Khan Academy – Modular Arithmetic** Great for understanding the basics of modular arithmetic, with step-by-step guides and visual explanations. Khan
- **Crypto101 – An Introduction to Cryptography** A free online book covering foundational cryptographic concepts, including modular arithmetic and finite fields.  
[Crypto101](#)

## 5.3 Polynomial Arithmetic

Polynomial arithmetic is the study of operations such as addition, subtraction, multiplication, and division of polynomials. In cryptography and coding theory, polynomial arithmetic is crucial, especially when the polynomials are defined over finite fields like  $\text{GF}(2)$  (binary field). This enables efficient and compact representation of mathematical operations for encryption and error correction.

### Polynomial Representation

A polynomial is an expression involving variables and coefficients, which are numbers (or elements of a field). For example, the polynomial  $p(x) = x^3 + 2x^2 + x + 1$  has coefficients  $\{2, 1, 1, 1\}$ . Polynomials in cryptography, particularly those in  $\text{GF}(2)$ , have binary coefficients (0 or 1), making them easier to work with in digital systems.

In  $\text{GF}(2)$ , each coefficient is reduced modulo 2. So the polynomial  $x^3 + x^2 + x + 1$  is written as  $x^3 + x^2 + x + 1$  (as the coefficients are already in  $\{0, 1\}$ ).

### Operations in Polynomial Arithmetic

#### 1. Polynomial Addition:

Polynomial addition is done by adding corresponding coefficients of the polynomials. In  $\text{GF}(2)$ , this operation is performed modulo 2. For example:

- $p(x) = x^3 + x^2 + 1$
- $q(x) = x^3 + x + 1$

To add these, align the coefficients and add modulo 2:

$$p(x) + q(x) = (x^3 + x^2 + 1) + (x^3 + x + 1) = x^2 + x$$

(The  $x^3$  terms cancel out and the constant terms also cancel out modulo 2.)

#### 2. Polynomial Multiplication:

Multiplying polynomials involves distributing terms and then combining like terms. For example:

- $p(x) = x + 1$
- $q(x) = x^2 + 1$

**Multiply them:**



$$p(x) \cdot q(x) = (x+1) \cdot (x^2+1) = x^3 + x + x^2 + 1$$

The result is  $x^3 + x^2 + x + 1$ , which is the product modulo 2.

### 3. Polynomial Division (Euclidean Algorithm):

Polynomial division is similar to long division for numbers. For example, divide  $p(x) = x^3 + x^2 + x + 1$  by  $q(x) = x^2 + x + 1$ .

The Euclidean algorithm helps us find the quotient and remainder when dividing one polynomial by another. In cryptography, polynomial division is often used to find the **inverse of polynomials** in finite fields, such as when working with AES encryption.

### Application in Cryptography

In cryptographic systems like AES (Advanced Encryption Standard), polynomials are used in the finite field  $GF(2^8)$ . This allows for efficient encryption and decryption operations, where polynomial arithmetic is used for byte substitution (using S-boxes), key expansion, and mixing operations.

Polynomial arithmetic is also essential in error-correcting codes, where operations over finite fields and polynomials help in correcting errors in data transmission.

## Next Steps

- **Study Polynomial Division & Euclidean Algorithm:** Focus on polynomial division and the Euclidean algorithm to find inverses in finite fields.
- **Explore  $GF(2^n)$  Finite Fields:** Understand finite fields like  $GF(2^n)$  and the role of irreducible polynomials in cryptography.
- **Implement Polynomial Arithmetic in Code:** Write code to perform polynomial operations (addition, multiplication, division) in  $GF(2)$  and  $GF(2^n)$  for cryptographic applications.

### References:

- "Mathematics of Public Key Cryptography" by Neal Koblitz: This book covers the theory of finite fields, including  $GF(2^n)$  and their application in cryptography. Mathematics of Public Key Cryptography
- A free online book introducing cryptographic concepts, including operations in  $GF(2^n)$  and their use in algorithms like AES. Crypto101

|     |                                                                  |   |
|-----|------------------------------------------------------------------|---|
| 5.2 | Advance Encryption Standard (AES): structure, key expansion..... | 3 |
|-----|------------------------------------------------------------------|---|

## 5.4 Advance Encryption Standard (AES): structure, key expansion

The Advanced Encryption Standard (AES) is a widely used symmetric encryption algorithm, designed to replace the older Data Encryption Standard (DES). AES operates on fixed-size blocks of data (128 bits) and supports key sizes of 128, 192, and 256 bits. It is widely used in various security protocols, including SSL/TLS and IPSec, and is the standard for data encryption in government and commercial applications.

### AES Structure:

AES is a block cipher, meaning it encrypts data in fixed-size blocks. The core structure of AES involves several operations that are repeated for a number of rounds, depending on the key size. AES works on a 4x4 matrix of bytes, which is referred to as the State. The number of rounds varies with the key size:

- AES-128: 10 rounds
- AES-192: 12 rounds
- AES-256: 14 rounds

Each round consists of the following four operations:

- **SubBytes:** Each byte of the state is replaced with a corresponding byte from the S-Box (Substitution Box). This operation provides non-linearity and helps in diffusing the plaintext.
- **ShiftRows:** The rows of the state matrix are shifted by a certain number of bytes. The first row is left unchanged, the second row is shifted by one byte, the third row by two bytes, and the fourth row by three bytes. This step provides diffusion.
- **MixColumns:** The columns of the state matrix are mixed to provide additional diffusion. This operation is not performed in the final round.
- **AddRoundKey:** A round key derived from the original key is XORed with the state. This operation ensures that each round has a different key and is crucial for security.

In the final round, the MixColumns step is omitted, but the other steps remain the same.

### Key Expansion in AES:

Key expansion is a process used to generate round keys from the original encryption key. The key expansion process starts with the original key (of length 128, 192, or 256 bits) and generates a series of round keys used in each round of AES encryption.

1. **Initial Key:** The original key is split into 4-byte words. The number of words depends on the key size:
  - **AES-128:** 4 words (16 bytes)
  - **AES-192:** 6 words (24 bytes)
  - **AES-256:** 8 words (32 bytes)
2. **Round Key Generation:** For each round, a new word is derived by applying a series of transformations:
  - **SubWord:** Apply the S-Box to the last word of the previous round.
  - **RotWord:** Rotate the word by one byte.
  - **Rcon:** XOR the word with a round constant, which changes depending on the round number.
  - **Round Keys:** These expanded keys are then XORed with the state during each round in the AddRoundKey step.

The key expansion ensures that each round uses a unique key, which increases the complexity and security of the encryption process.

AES is a highly secure and efficient encryption algorithm with a structured process involving multiple rounds of substitution, shifting, mixing, and key expansion. Its flexibility with key sizes makes it adaptable for various security requirements.

## Next Steps

- **Study AES Decryption Process:** Learn how the AES decryption process reverses the encryption steps (InvSubBytes, InvShiftRows, InvMixColumns) and uses round keys in reverse order.
- **Investigate AES Security and Attacks:** Explore AES's resistance to cryptanalysis and side-channel attacks. Study how key size impacts AES security (128, 192, 256 bits).

**References:**

- **NIST FIPS PUB 197 – The Official AES Standard:** Describes both encryption and decryption algorithms in detail. [Link to AES Specification \(FIPS 197\)](#)
- **"Cryptography and Network Security" by William Stallings**

**Parul<sup>®</sup> University** **NAAC** **A++**  
Vadodara, Gujarat GRADE



<https://paruluniversity.ac.in/>

